

Chapter - 2 (340)

* X0 register is also used to copy values among registers [Add x_6 , x_5 , x_0]
(copies x_5 to x_6)

* RISC-V has a modular design, meaning it has a base ISA and also has optional extension which allows the designers to design application specific instructions which specifically tailored to fulfill specific

needs such as optimizing performance, increase power efficiency for their target workload. These extensions are designed over the base ISA

- * That customized instructions actually improves performance

- * 32×64 architecture is the most optimum size that does not hamper the desired clock period $\downarrow$ (on increase)

- * Smaller is faster

↳ If system has less register then access time decreases

* Global Pointer: Points to area in memory where global and static variables are stored, specific area gives faster access to data as the base address to that area is stored in global pointer variable

* Thread Pointer: This register holds address to thread specific data such as thread local variables, thread control blocks (TCB).

* Thread pointer register

Program Memory Layout

* Stack (frames, local variable) → stack pointer

* Heap (dynamic variables which does not have fixed size at runtime)

* Thread-local storage → thread pointer

* Global / Static data → global pointer

Code (text segment)

(compiled machine code)

provides quick way for each thread to find its data

- * Different threads will have different memory region

- * Frame pointer points to data that are specific to current function's stack frame in stack, frame pointer will be different for different function calls

- * Stack frame: stores current function's local variables, saved registers etc.

$$* F = \underbrace{(a+b)} - \underbrace{(c+d)}$$

Results of these intermediary operations are stored in temporary registers as they are not the main goal, they are just steps to achieve main goal

* $x_{10} - x_{11}$ or $x_{12} - x_{17}$ stores the values of the arguments that we pass to call a function

* Each memory cell stores 8-bit data

* For an array A of

length 3, $A[0]$ will be stored in 8 consecutive memory cell as data is 64-bit, so $A[0]$ will be stored in from memory cell 0000 to 0007, $A[1]$ will be stored in from 0008 to 0015

* $A[3] = ?$

Formula $\frac{8 * \text{index}}{\text{offset}} + \text{base}$

$$8 \times 3 = 24 + \text{base} = 24 + 0000 \\ = 0024$$

$A[3] \rightarrow 0024 \text{ to } 0031$

* Big Endian: stores MSB in lowest location and the order goes on till LSB gets stored in highest location

* RISC V is little Endian
(Reverse of big endian,
LSB byte stored in lowest memory location and so on

* Little endian approach is used by RISC V while

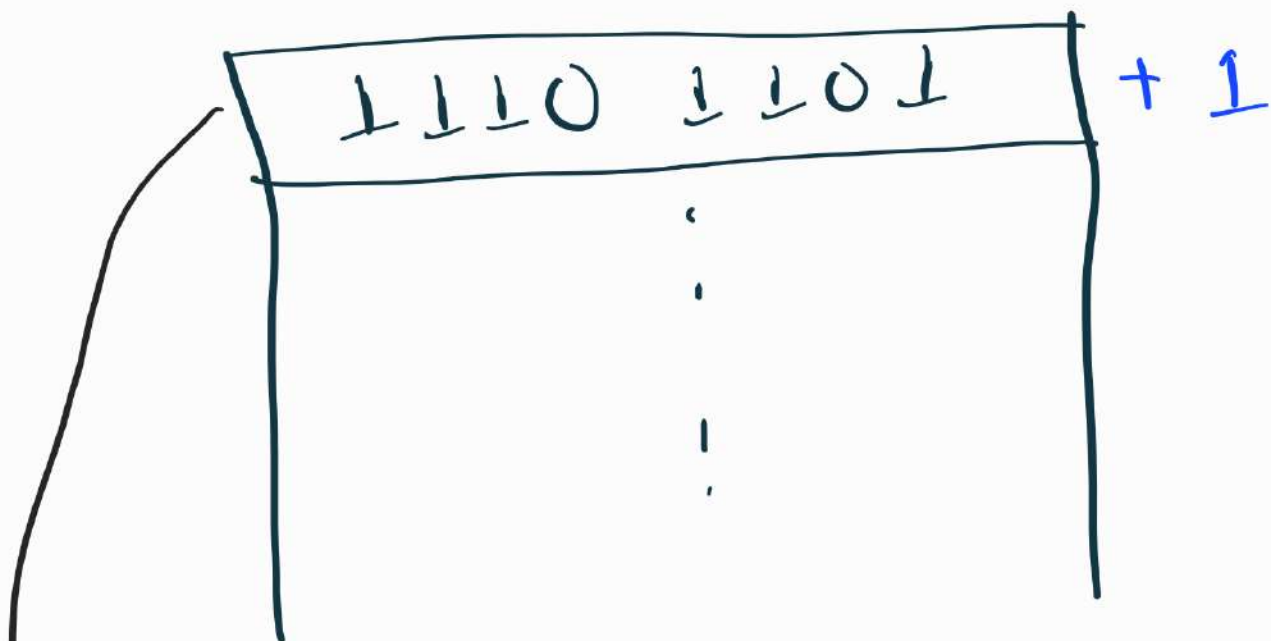
storing data into memory

... 1 1 1 0 1 1 0 1
 + 1

... 1 1 1 0 1 1 1 0 → 1 byte

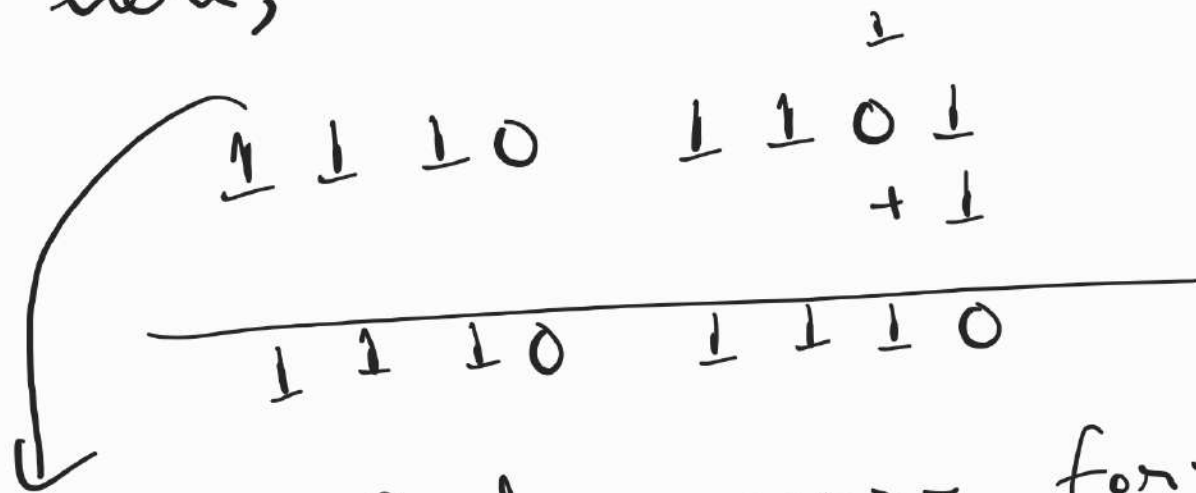
↓
Unaffected in this operation

In little endian approach



→ we only need to load
this data byte, no need
to load others as they

are not affected
we load only one byte
here,



If we had a carry forward
here then we would need to
load the next byte

* For big endian we would
need to load the whole
data to perform operation

* Base address is stored in
base register

* No alignment
restriction in
RISC-✓

* $\text{ld } X_{21}, 64(X_{20})$

↓
Loading data
into this register

↓
base register
↓
offset (we
will get the
data if we go
64 more cells
from base address)

Register
destination
→ $X_{21}, 64(X_{20})$
↑
Register
source

if $X_{20} = 0000$
then data starts
from,
 $0000 + 64$
 $= 0064$

* $\text{sd } X_{21}, 40(X_{20})$

Basically, $A[8] = A[5]$

$A[8]$ has offset 64 and

$A[5]$ has offset 40

x_{21} , 40 (x_{20}) $\rightarrow$ This is not destination
 $\downarrow$
Register
Source 1
Source 2
(As we calculate the address using this value)

* $g = x_{20}$, $h = x_{21}$,

Base of $A = x_{22}$,

A is a double word array
of length 100

$$g = h + \underbrace{A[8]}_{\text{It is in memory}};$$

ld $x_5, 64(x_{22})$
 $\uparrow$ offset $\hookrightarrow$ base reg

Add x_{20}, x_{21}, x_5

$h = x_{21}$, Base of $A = x_{22}$

$A[12] = h + A[8];$

ld $x_5, 64(x_{22})$

Add x_5, x_{21}, x_5

sd $x_5, 96(x_{22})$

* Immediate operand:

we perform operations using constants / immediate value

Addi $\rightarrow$ Add immediate

addi dst , source 1 , source 2
(constant)

* Compiler can't add two integers

addi x21, 4, 5

$\nearrow$ Not allowed,
 $\downarrow$
This position should be a register

* There is no subi

Do, Addi x22, x22, -5

* Immediate operands avoid a load instruction

`addi x21, x21, 5`

↓
This 5 is hardwired into the instruction (No need to load this)

* So it is faster

* LB → Load Byte,

We are loading 8-bit data but the register can store 64-bit, so we need to extend the data

* Two types of extensions
i) Signed ii) Unsigned

Unsigned

$$(2)_{10} = (10)_2$$

2 in 8-bits unsigned

$$= (00000010)_2$$

LBU X_{22} , $\frac{10(X_{21})}{\text{Memory location}}$
↓ where 2 is stored in 8-bit

This instruction will load the 8-bit data and extend it to 64-bit in unsigned format

$$(+2)_{10} = 010$$

↓
sign bit

extend to 8-bit,

Copy the sign bit,

00000
Copied
sign bit

010
Actual number

$$(-2)_{10} = \begin{array}{r} 1\ 1\ 1\ 1\ 1\ 1\ 0\ 1\ (1's) \\ +\ 1 \\ \hline 1\ 1\ 1\ 1\ 1\ 1\ 1\ 0\ (2's) \end{array}$$

LB $\rightarrow$ Performs signed extension, does 2's complement for negative numbers

Unsigned range,

$$0 \text{ to } 2^n - 1$$

Signed range for 2's comp,

$$-2^{n-1} \text{ to } +(2^{n-1}) - 1$$

* Desire to keep instruction format same,

Add $\underbrace{x_{20}}_{\substack{\text{reg} \\ \text{des}}}, \underbrace{x_{20}}_{\substack{\text{reg} \\ s_1}}, \underbrace{x_{21}}_{\substack{\text{reg} \\ s_2}} \} 1 \text{ format}$

Addi $\underbrace{x_{20}}_{\substack{\text{reg} \\ \text{des}}}, \underbrace{x_{20}}_{\substack{\text{reg} \\ s}}, \underbrace{5}_{\text{immediate}} \} \text{Same format}$

ld $\underbrace{x_{20}}_{\substack{\text{reg} \\ \text{des}}}, \underbrace{16}_{\text{imm}} \left(\underbrace{x_{21}}_{\substack{\text{reg} \\ s}} \right)$

sd $\underbrace{x_{20}}_{\substack{\text{reg} \\ s_1}}, \underbrace{16}_{\text{imm}} \left(\underbrace{x_{21}}_{\substack{\text{reg} \\ s_2}} \right) \} \text{Another format}$

* It is not possible to all

instructions to have same format

* That is why length of each instruction is kept fixed

* If format were kept same then length would have varied

3 formats taught in this course,

i) R Type

↳ Uses exactly 3 registers

1 → Reg Destination

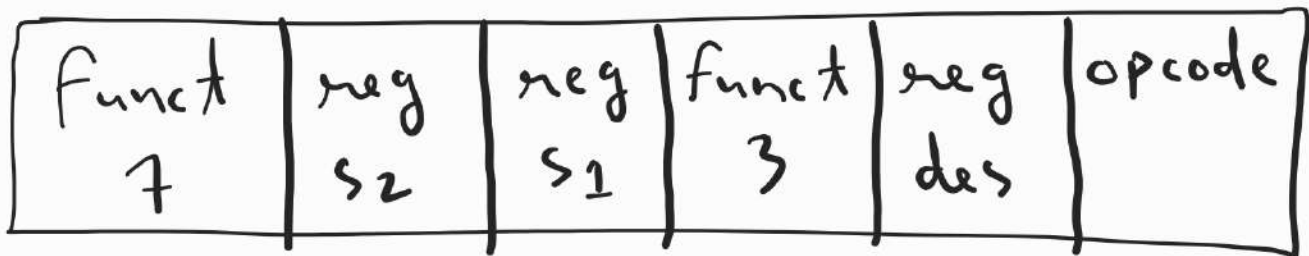
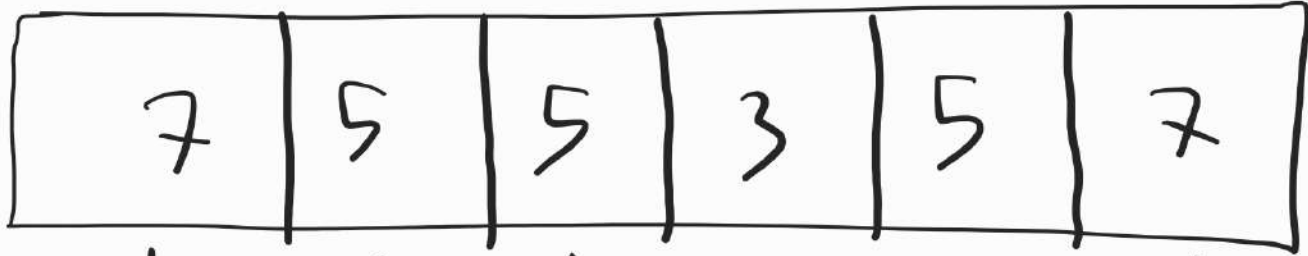
2 → Reg Source

Ex : Add, Sub, SLL, AND, OR, XOR etc.

R type ins fields

31

0



7-bit

6 fields in total

opcode → partially denotes
the format of instruction

* opcode, funct 3 and 7
together denotes the
instruction type and functionality

* For R Type, opcode tells
the instruction is of type R

* Reg s_1 , s_2 and des are 5-bit because there are 32 registers, X_0 to X_{31}
 $(31)_{10} \rightarrow (11111)_2$, needs exactly 5-bit

* Opcode will be given in question, directly or in a table or in hex format

* If $funct3$ and $funct7$ are not given then put 0 there

Ex,

Add X_{20} , X_{20} , X_{21}

Machine code is below,

0000 000	10101	10100	000	10100	1001001
-------------	-------	-------	-----	-------	---------

R	1001001	} Opcode
I		
S		

* If it is mentioned that we need to convert the instruction to hex then we convert the 32-bit machine code to hex by grouping 4-bits each

ii) I type instruction

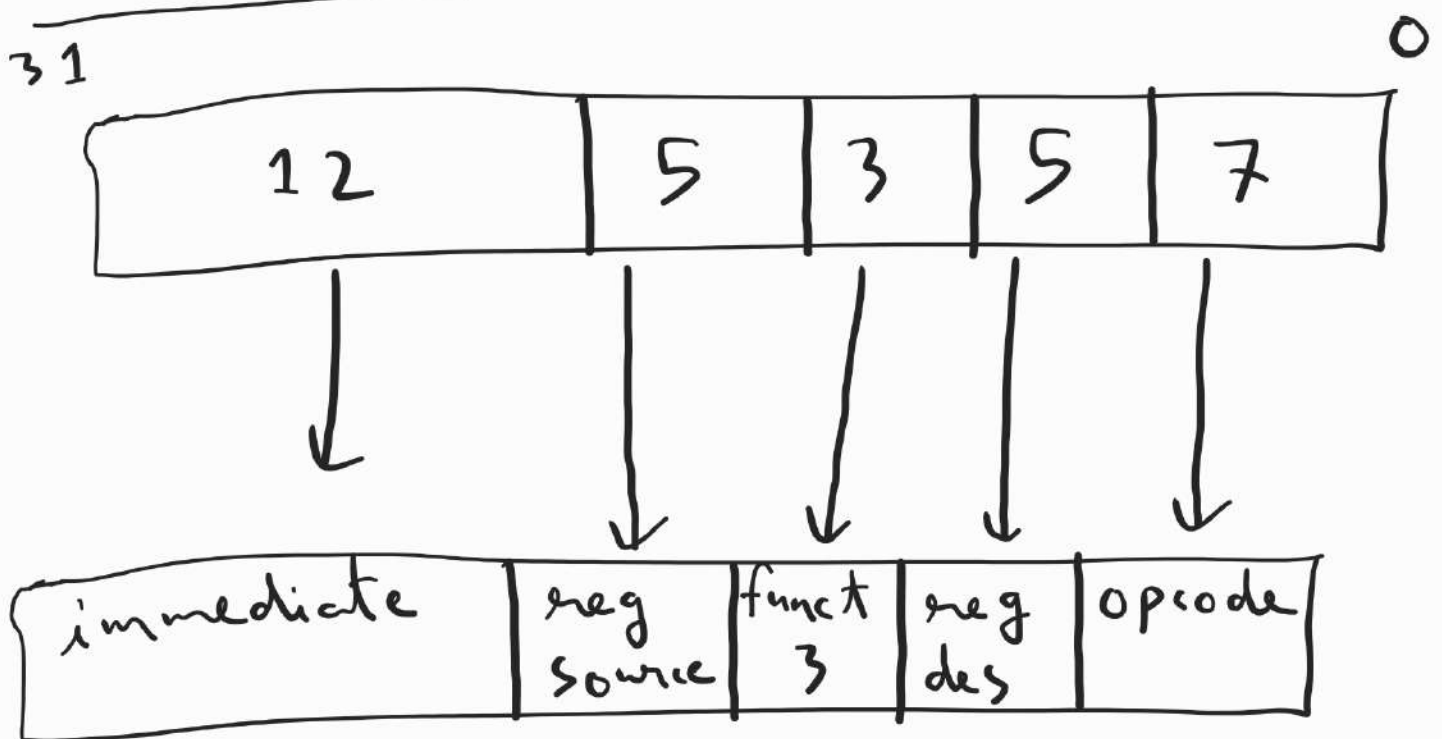
↳ should have 1 immediate value

↳ must have 2 registers

↳ reg des

↳ reg source

Format



immediate → constant/offset

reg s1 → source/base register

funct 3 $\rightarrow$ width + signedness
(For load instruction)

width $\rightarrow$ size of data to be loaded

signedness $\rightarrow$ data is signed or unsigned

LB	00	2-bit for size
LH	01	
LW	10	0/1 for sign
LD	11	
		total $\rightarrow$ 3-bit

* Ex: 6 represents the width and signedness of load inst

Table

000
⋮
110

(LW)

$\rightarrow$ figure the instruction out

S Type Instruction

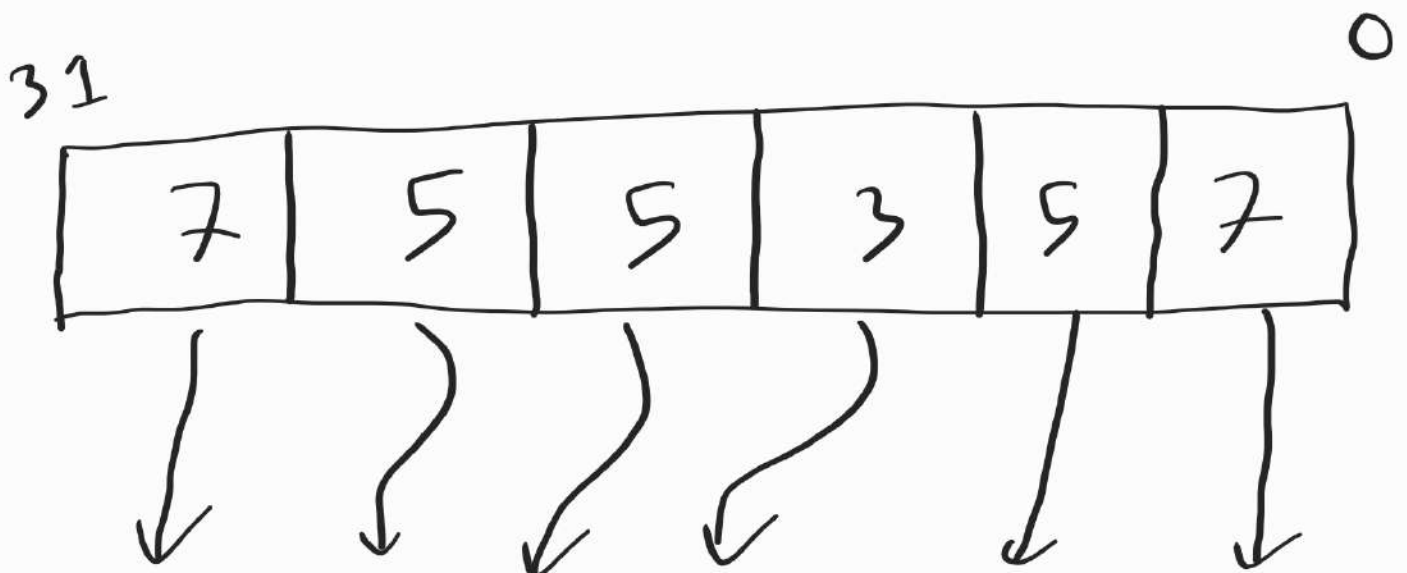
* Store instruction is only S type instruction, no one else

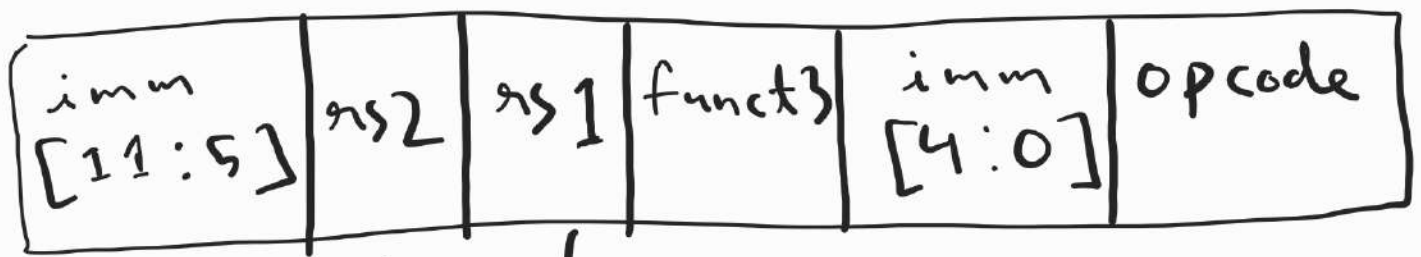
* Variants of store

SB
SH
SW
SD

} Signed Store Inst

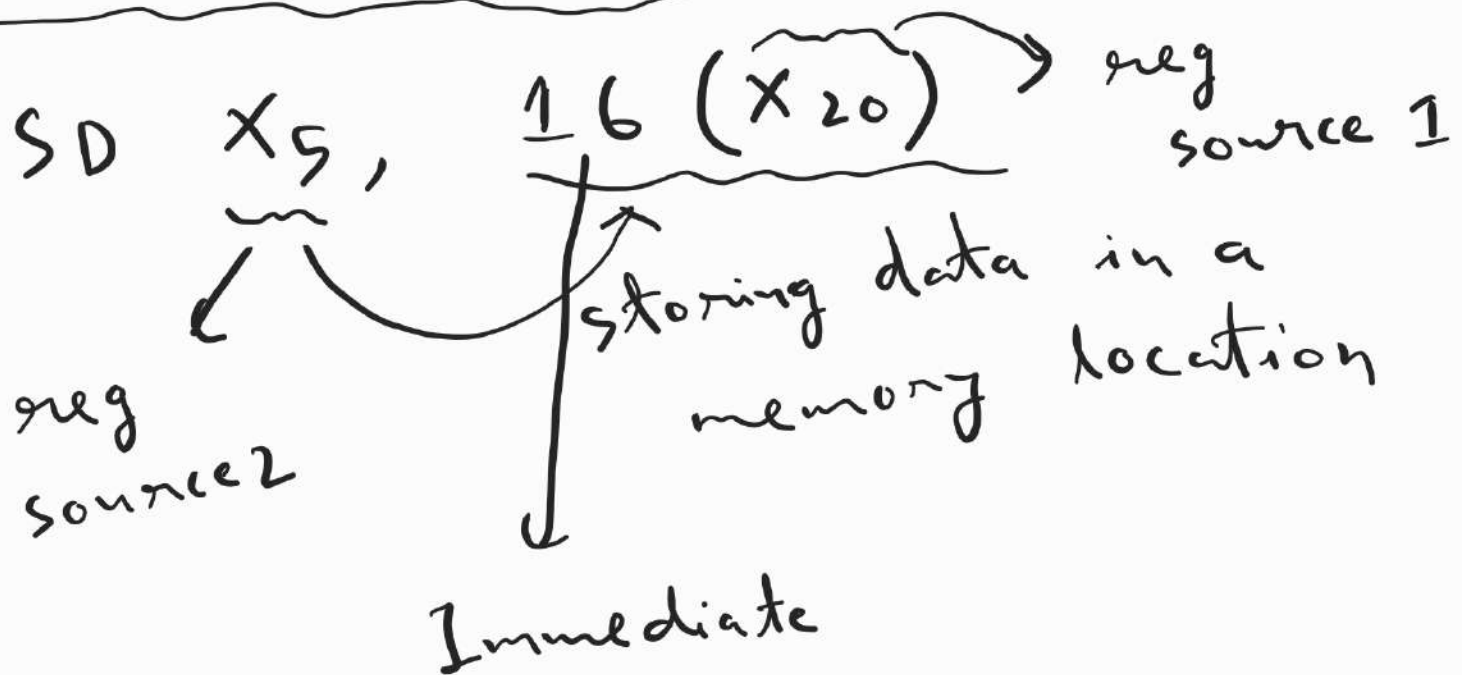
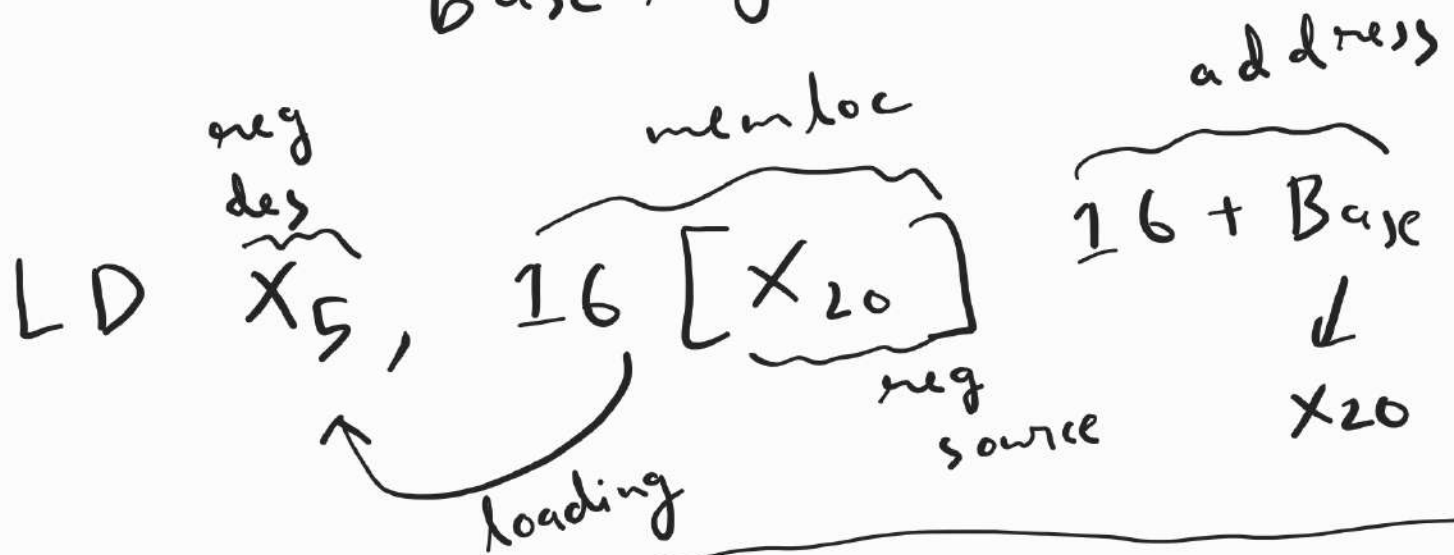
* There are unsigned as well





↓
source

↓
base reg

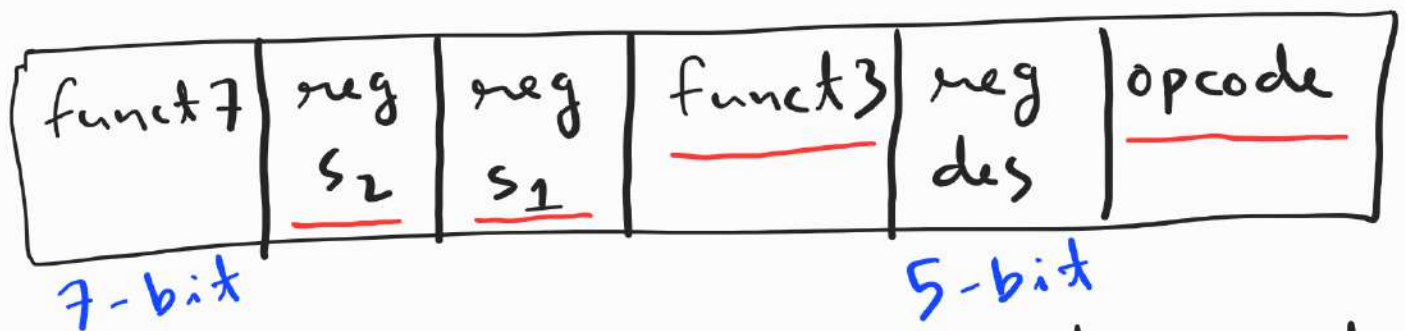


* we wanted to keep the size of instructions same in the machine code

* we also try to keep the format similar for all types of instructions if possible

Let's look at R type ins format,

→ R type

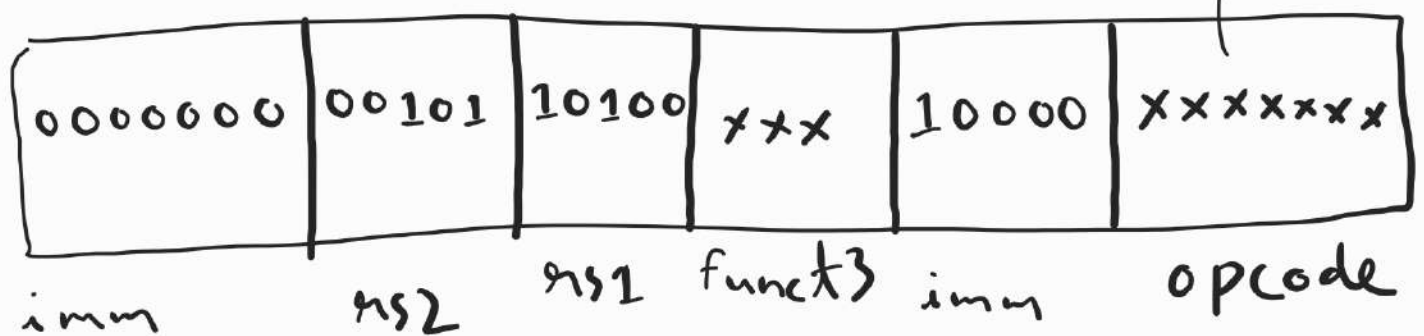


* Position of the red marked fields are at the same position as S type format

* Immediate bits are split into two portions because in R type the blue marked

fields have the same capacity (5-bit and 7-bit), to keep both format similar to some extent, we split the immediate bits into two portions (5-bit and 7-bit) in S type and put them in a similar position

* SD $x_5, 16(x_{20})$ In 12-bits
 $\swarrow$ $\searrow$
 rs_2 rs_1 $\rightarrow$ Any



[11:5]

bit 11 to bit 5

here (7-bit)

[4:0]

bit 4 to bit 0

here (5-bit)

* funct3 is same as load instruction, weight and signedness

Shift operation

* 1010 1111

Left shift 1 time,

0101 1110 → vacant space filled with 0
(not rotate)

↓ Left shift again by 1

10111100 (2 times left shift done from original number)

Right Shift,

1011 1100

0101 1110

1

↓
vacant filled with 0

Sll → Shift left logical

Slli → " " " i

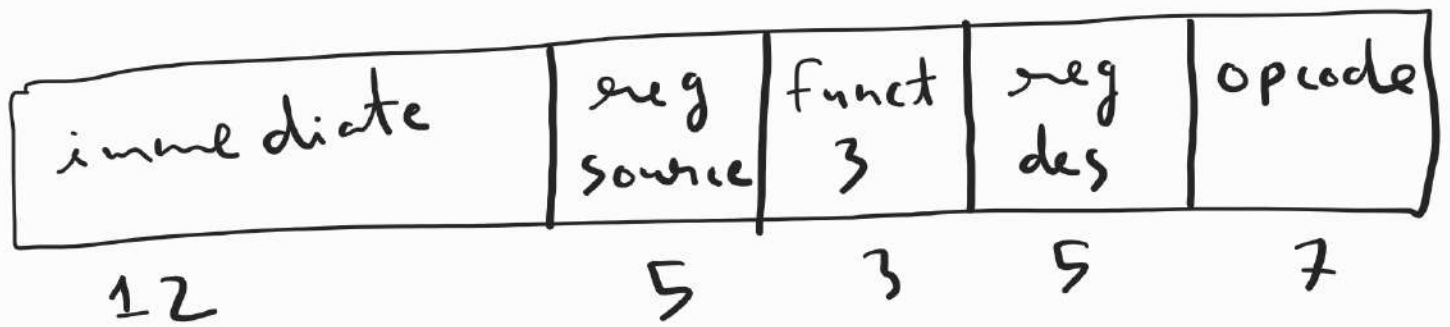
Slli x_{19} , x_{20} , 4 → 4 time
 ↓ ↓ ↓ left
 reg reg imm shift
 dest source the value
 in source
 and store
 in dest

* Slli is I type instruction
as it matches the format

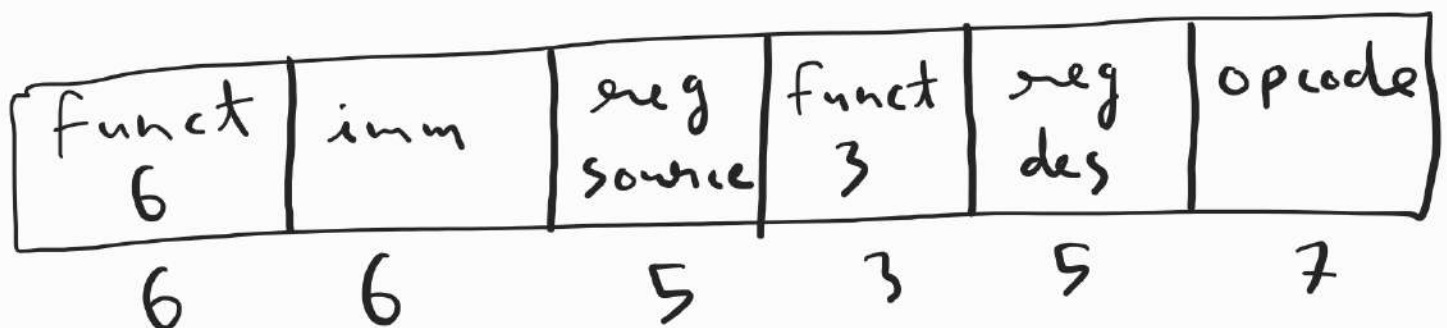
* If we perform left/right
shift 64 times in RISC-V
the value becomes zero as

the capacity of the registers are 64-bit in RISC-V, that is why the value of the immediate parameter is capped from 1 to 63

I type format



Difference in shift machine code,



* Immediate is broken into two 6-bit parts as the valid range for shifting is 63 and it takes 6-bit to write 63, that's the immediate

* In left shift, we can perform multiplication by 2^i if we shift i times

$\text{Sll}i\ X_{11}, X_{19}, 4$

* The value which will be stored in X_{11} will be

$X_{19} * 2^4$ since we performed left shift 4 times

8-bit data,

$$0000 \ 1100 \rightarrow 12$$

$$0001 \ 1000 \rightarrow 24 \quad (12 * 2^1)$$

1-bit left shift

$$0011 \ 0000 \rightarrow 48$$

$$\hookrightarrow (12 * 2^2)$$

2-bit left shift

* Limitation:

$\hookrightarrow$ Can only multiply with numbers that are multiples of 2

Solution:

12×5 can be written

as $(12 \times 4) + 12$

$\hookrightarrow$ multiple of 2

* If we want to multiply with a number that is not a multiple of 2 then we will go close to that number which is a multiple of 2 and add/sub rest the remaining numbers

Ex: $12 \times 7 \rightarrow (12 \times 8) - 12$

* For right shift

We can perform division by 2^i if we perform right shift i times

Sol: $X_{11}, X_{19}, 4$

* X_{11} will store $X_{19}/2^4$

* Same procedure for dividing with a value that is not multiple of 2

AND Operation

* AND is done for bit masking purpose

↳ we select and copy some bits and make other bits zero

A	B	A.B
0	0	0
1	0	0
0	1	0
1	1	1

$x.0 = 0$
 $x.1 = x$

and X_9 , X_{10} , X_{11}

$\underbrace{\hspace{1cm}}_{\text{reg des}}$
 $\underbrace{\hspace{1cm}}_{\text{reg s1}}$
 $\underbrace{\hspace{1cm}}_{\text{reg s2}}$

* AND is done bitwise between data

$X_{10} = \dots 1001$
 $X_{11} = \dots 1100$
 $X_9 = \dots 1000$

$\uparrow \uparrow \rightarrow$ bitwise and

* This is an R type instruction

$X_{10} = \dots 0000$

$\begin{matrix} 7 & 6 & 5 & 4 & 3 & 2 & 1 & 0 \\ \uparrow & \uparrow & \uparrow & \uparrow & \uparrow & \uparrow & \uparrow & \rightarrow \end{matrix}$
 $\begin{matrix} 1100 & 1100 \end{matrix}$

$X_{11} = \dots 0000$

$\begin{matrix} 1100 & 0100 \end{matrix}$


 Wrong, we don't know what's in data

Correct, $\rightarrow$ This is called the mask,
we use this to copy and discard

$$X_{11} = \dots\dots\dots 0000 \ 1111 \ 0110$$

* We put 0 in positions where
we don't want to copy bits,
put 1 where we want to
copy bits as $X \cdot 1 = X$

[Keep (7-4, 2, 1) positioned
bits same (copy the bits) and make
rest zero for X_{10}] Question

$$X_9 = \dots\dots\dots 0000 \ 1100 \ 0100$$

$\rightarrow$ This is the result
after copying

* OR $\rightarrow$ Include bits

OR $\underbrace{X_9}_{\text{reg des}}, \underbrace{X_{10}}_{\text{reg s}_1}, \underbrace{X_{11}}_{\text{reg s}_2}$

Setting bits as 1

* R type instruction

A	B	A + B
0	0	0
1	0	1
0	1	1
1	1	1

$X + 0 = X$

$X + 1 = 1$

Set these bits as 1, rest are same

$X_{10} = 0000 \dots 0000 \ 1100 \ 0000$

$X_{11} = 0000 \dots 0000 \ 0011 \ 0000$

$X_9 = 0000 \dots 0000 \ 1111 \ 0000$

* NOT is not directly there
 * we have XOR

A	B	$A \oplus B$	
0	0	0	} XOR with 0 $\rightarrow$ buffer
1	0	1	
0	1	1	} XOR with 1 $\rightarrow$ NOT
1	1	0	

$$X \oplus 0 = X$$

$$X \oplus 1 = X'$$

XOR X_9 , X_{10} , X_{11}
 reg reg reg
 des s1 s2

* R type instruction

$$X_{10} = \dots 0000 \underbrace{1011}_{\text{10}} 0000$$

$X_{11} = \dots \dots \underline{0000 \ 0011 \ 0000}$
mask

$$X_9 = \begin{matrix} & & & 0000 & 1000 & 0000 \\ . & - & . & & & \end{matrix}$$

$$A[3] = A[5] + 2$$

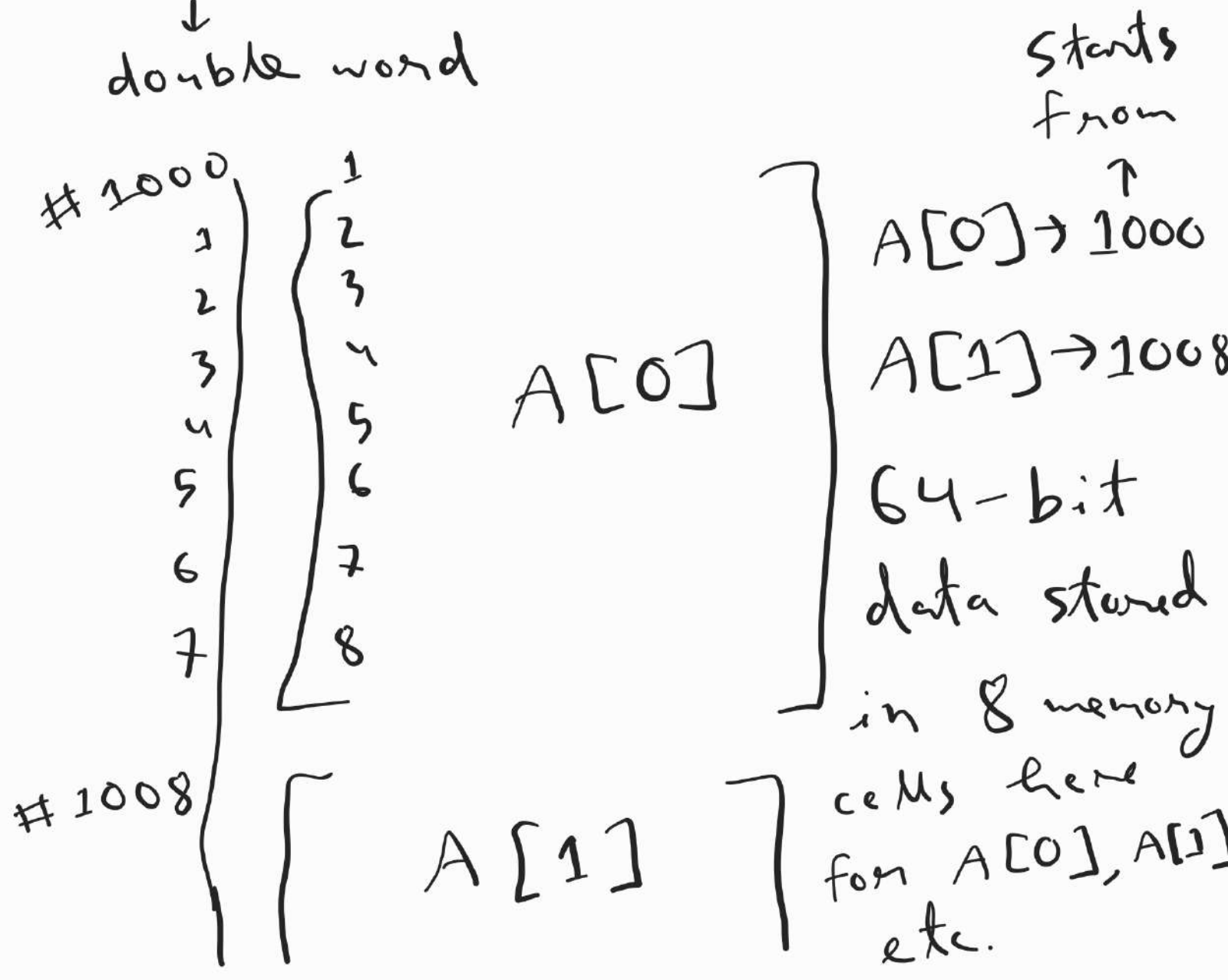
$$A[0] = X_{21} \quad (\text{Base Register})$$

A is a double word array

* Base address of A is stored in x_{21} , making it base register

* Memory is byte addressed,
 since A is double word
 so one data will take
 8 cells in memory as
 data is 64-bit here

LD X5, 40(X21)
 ↓
 double word



$A[0] \rightarrow 1000 \text{ (Base)}$

$$A[1] \rightarrow 1008 = \text{Base} + 8 \times 1$$

$$A[2] \rightarrow 1016 = \text{Base} + 8 \times 2$$

For word (32-bit)

4 cells to store the word
as each memory cell stores
8-bit,

$A[0] \rightarrow 1000 \text{ (Base)}$

$A[1] \rightarrow 1004 = \text{Base} + 4 \times 1$

$A[2] \rightarrow 1008 = \text{Base} + 4 \times 2$

LD X5, 40(X21)

Add: $x_6, x_5, 1$

SD $X_6, 24(X_{21})$

Now, $A[g]$ instead of
 $A[5]$ where $g \rightarrow X_{25}$
(g is associated with X_{25})

* g is a variable hence it
is subjected to change,
we need to do $g \times 8$

$A[g]$
 $\downarrow$
 $\swarrow$ $g \times 8$
 $= g \times 2^3$

SM: $X_5, X_{25}, 3$
 $\downarrow$

X_5 has the calculated offset

LD $X_6, \underline{\quad\quad\quad}(X_{21})$???

$\rightarrow$ we can't write
 X_5 here

* X_5 is a register and LD instruction requires an immediate value in that position according to syntax

So,

Add X_6 , X_{21} , X_5
 $\uparrow$ $\uparrow$
 Base offset

$X_{21} + X_5 \rightarrow$ Actual location
(we are calculating the address)

Now,

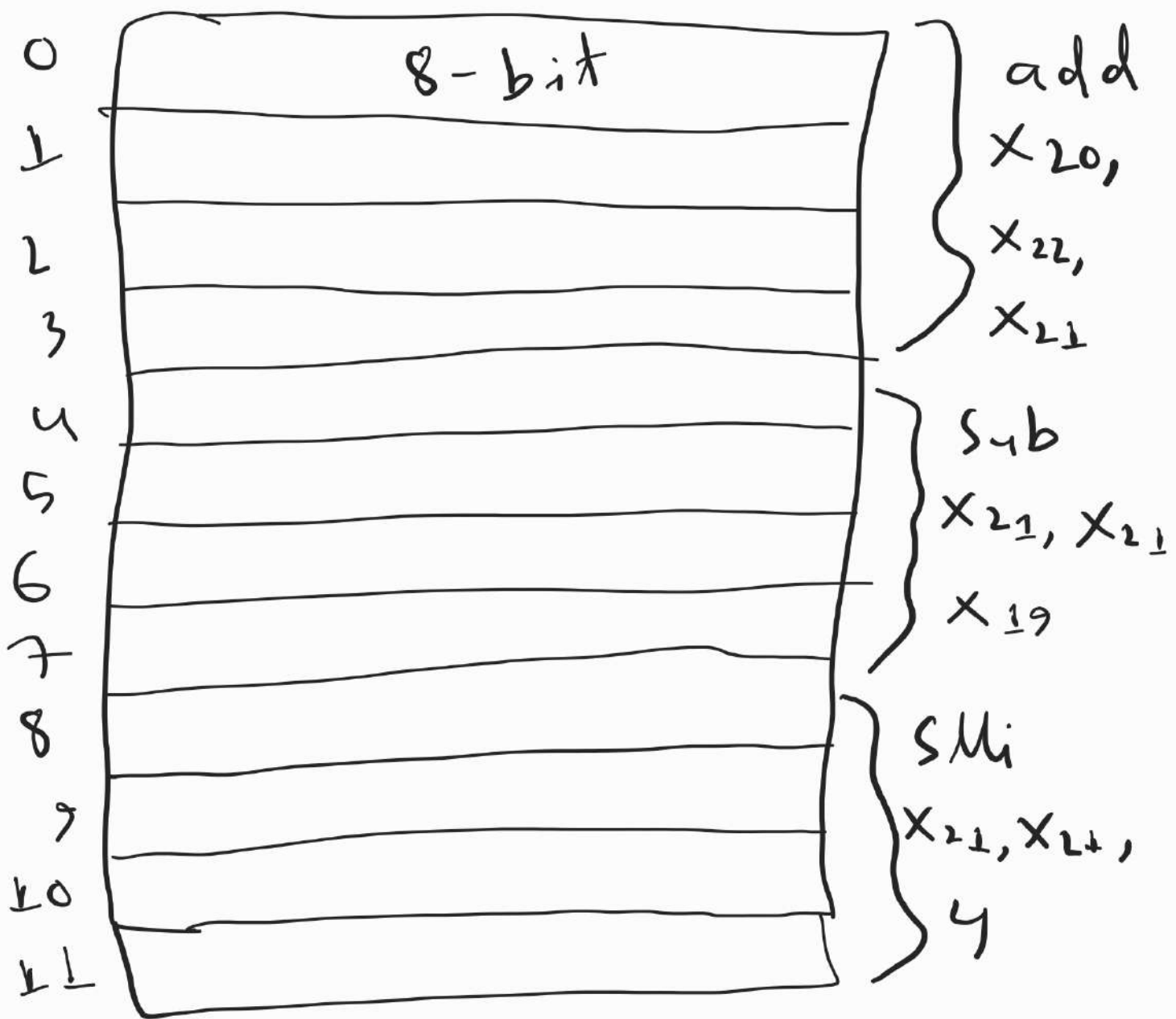
LD X_7 , $0(X_6)$
 $\swarrow$ $\rightarrow X_6$ has the address
 No offset
needed as we
calculated the address

Branching / Condition / Decision Making

- * Assembler runs a portion of assembly code sequentially
- * Each machine instruction is 32-bit hence it will take 4 memory cells to store a machine instruction as one memory cell stores 8-bit

add $\rightarrow$ 0
sub $\rightarrow$ 4
slli $\rightarrow$ 8

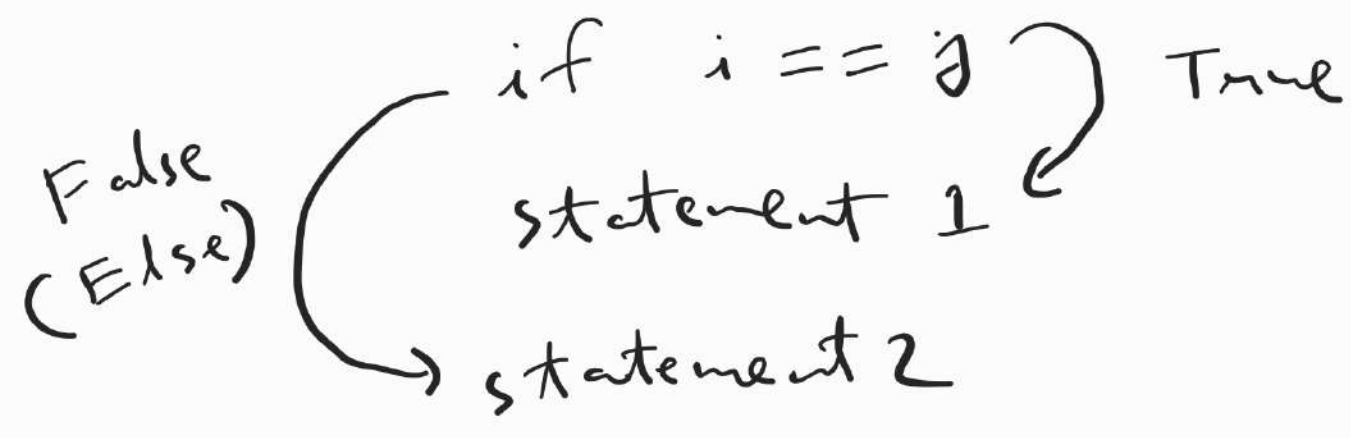
(starts from)



* Assembler will run the instructions sequentially, no skip in between

* But we can instruct the assembler to skip and that

is how branching is done



* We need the assembler to skip to statement 2 if the condition does not match

* 2 decision making instructions

↳ beq rs1, rs2, L1
(branch if equal) ↑
 label

* If the comparison gives

true then it jumps to the label

↳ bne rs1, rs2, L1
(Branch if not equal) ↑
 label

* Both statements are called conditional branch as they decide whether to jump or not based on certain condition

* Formal definition: testing a value, based on test result allows for a transfer of control to a new address in the program

Conditional Jumps

i) Sign $\rightarrow$ =

Inst $\rightarrow$ beq, beq rs1, rs2, L1
operation $\rightarrow$ rs1 == rs2

ii) Sign $\rightarrow$!=

Inst $\rightarrow$ bne, bne rs1, rs2, L2
operation $\rightarrow$ rs1 != rs2

iii) Sign $\rightarrow$ <

Inst $\rightarrow$ blt, blt rs1, rs2, L3
operation $\rightarrow$ rs1 < rs2

iv) Sign $\rightarrow$ >=

Inst $\rightarrow$ bge, bge rs1, rs2, L4
operation $\rightarrow$ rs1 >= rs2

if (i == j) :	f = x ₁₉
f = g + h	g = x ₂₀
else :	h = x ₂₁
f = g - h	i = x ₂₂
	j = x ₂₃

* we try to use bne because that helps us skip statement more conveniently

bne x₂₂, x₂₃, Else

add x₁₉, x₂₀, x₂₁

bne x₀, x₀, Exit

Always true (Unconditional branch)

Else :

sub x₁₉, x₂₀, x₂₁

Exit :

* Label is just a name but it needs to be unique, should not start with number
(Follow variable naming format)

* Why not write beq at the beginning?

```
if (i == j) {
```

```
    f = g + h;
```

```
}
```

beq	x22, x23,	If	} More code needed here (using beq)
beq	x0, x0,	Exit	
If:			
Add	x19, x20, x21		
Exit:			

bne x22, x23, Exit	} Using bne
Add x19, x20, x21	
Exit:	

if ($a \leq b$):

$f = g + h$

else:

$z[g] = z[5] + h$

bge a, b $\rightarrow b \geq a$ (Reverse it)

bge x22, x21, If

lw x5, 20(x26)
↳ base

add x5, x5, x25

slli x6, x24, 2

a = x21

b = x22

f = x23

g = x24

h = x25

z[0] = x26

z
↳ word
Array

bge b, a
denotes

$b \geq a$,
interchanged
sources

add x_6, x_6, x_{26}

sw $x_5, 0(x_6)$

beq x_0, x_0, Exit

If:

add x_{23}, x_{24}, x_{25}

Exit:

Loop

while ($\text{save}[i] == k$)

$a = a + 1$

$i = i + 1$

Loop:

ld $x_9, 0(x_{25})$

(loop starts from
0th index here)

$\left[\begin{array}{l} i = x_{22} \\ k = x_{24} \\ a = x_{23} \\ \text{save}[0] = x_{25} \end{array} \right.$

bne X9, X24, Exit

addi X23, X23, 1

addi X22, X22, 1

beq X0, X0, Loop

Exit:

- * This loop will always load save [0] in X9 hence won't access other values

- * Offset remains static in this code

- * We need to calculate the offset before using some other instructions

Correct Code:

Slli x22, x22, 3 $[i = i * 2^3]$

(More Problem here)

(i will store 8 after it becomes 1 as $i = i * 2^3 = 1 * 2^3 = 8$, iterating variable won't work)

Now,

Loop:

Slli x10, x22, 3

Add x10, x10, x25

ld x9, 0(x10)

bne x9, x24, Exit

addi x23, x23, 1

addi x22, x22, 1

beq x0, x0, Loop

Exit:

$x_{19} \rightarrow \underbrace{1111\ 1111\ 1111\ 0000}_{16\text{-bits}}$

Store this value in x_{19} ,

Can't use

addi $x_{19}, x_0, 65520$

as immediate is of 12 bits

in I type instruction

* We use LUI here,

Load Upper Immediate

↳ It forms 32-bit immediate

LVI and, constant $\rightarrow$ 20-bit

Load a 64-bit value in x_{19} ,

* LUI copies 20-bit

data into rd's $[31:12]$

So, it will identify bit 12 to bit 31 in the 64-bit numbers and convert it to decimal and copy

Ex:

$\dots 0000 \quad 0000 \quad 0000 \quad 0011 \quad 1101 \quad 0000 \dots$
 $\quad \quad \quad \underbrace{\hspace{10em}}_{12}$
 $\quad \quad \quad \underbrace{\hspace{10em}}_{976}$

copy these bits to X_{19} and
then extend left bits with
bit 31's value, which is 0
here

* So basically copy whatever there is in bit 31 to bits [63:32]

* And copy 0's in [11:0] bits

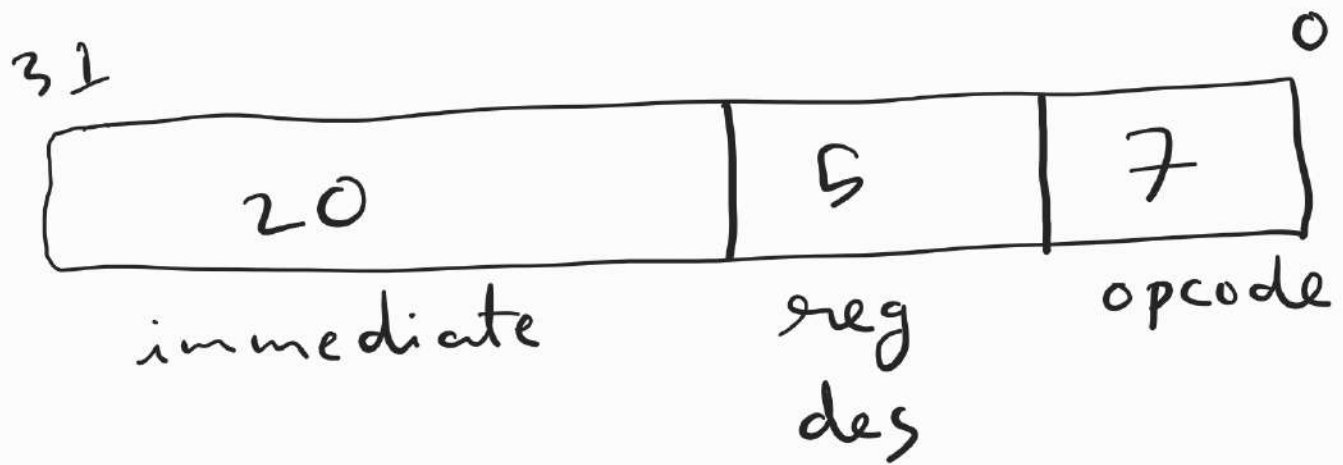
* leftovers in bit 0 to bit 11, this 12-bit value is added later to the previously loaded value using addi (Actual number is 3998976)

lui X19, 976

addi X19, X19, 1280

(Remaining values in bit 0 to 11)

* LUI is U type inst



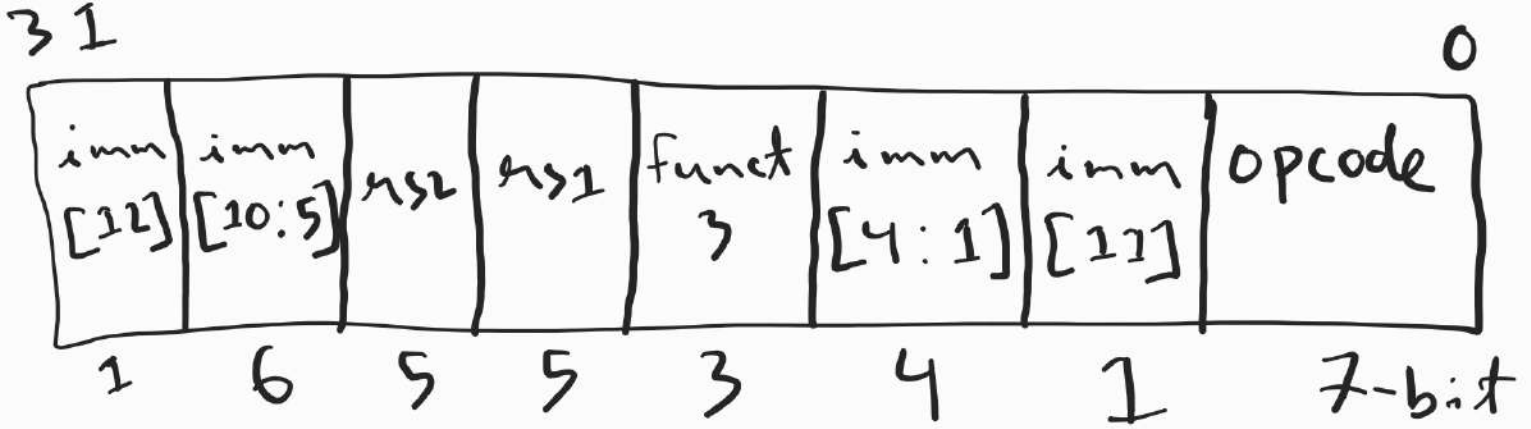
SB type instruction

↳ beq, bne, blt, bge

Jal → Jump and link
(Unconditional Jump)

(Used in procedure calling)

SB type format



* It has a 12-bit immediate value broken into multiple fields

Rule 1]: The value of the immediate in this format has to be a multiple of 2, the immediate here represents branch address

Rule 2]: Allows forward and backward moving

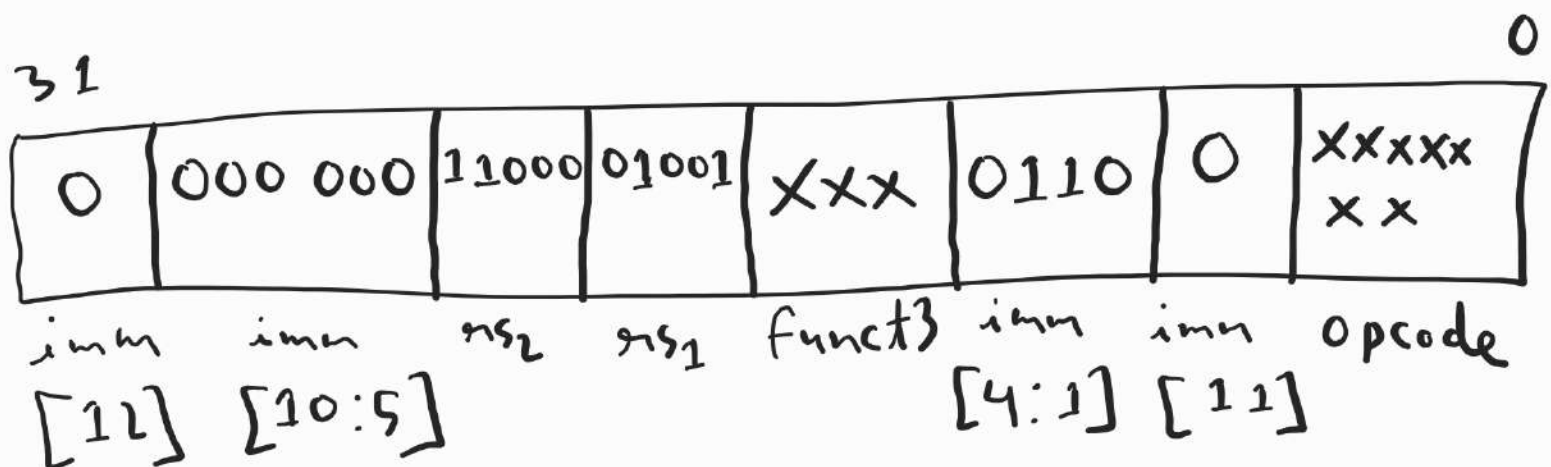
location	Loop;	← backward moving
# 80000	slli x10, x22, 3	line 1
# 80004	add x10, x10, x25	line 2
# 80008	ld x9, 0(x10)	line 3
# 80012	bne x9, x24, Exit	4
# 80016	addi x22, x22, 1	lines
# 80020	beq x0, x0, Loop	6
	Exit:	← Forward moving
# 80024		line 7

* we declare a line after the Exit

Rule 3]:

The unusual encoding of immediate is done because it makes the datapath simpler (More in chapter 4)

* Machine code for line 4 in above code:



bne X9, X24, Exit

* We need to figure out an offset corresponding to the

label

* That offset is called
PC Relative Addressing

↳ $PC + \text{immediate} \times 2$

↓
(Program
Counter)

↓
* There is a register which
stores the location of
the instruction that is
being executed currently

* We need to figure out
how much we are jumping
from the current line

* The base address is the PC

* Exit is a label here, that is why we need to declare another line after

* We are jumping from line 4 to line 7, we have 3 lines in between including the line we are jumping to

Now

$$3 \times 4 = 12$$

↓
No of
lines in
between

↓
No of
slots
needed in
memory to store an
instruction

↳ we convert
this to a 12-bit
binary number

0000 0000 1100 (Step 1)

0 0000 0000 1100 (Step 2)

↓ ↓ 10:5 4:1 ↘ Discard

12 11

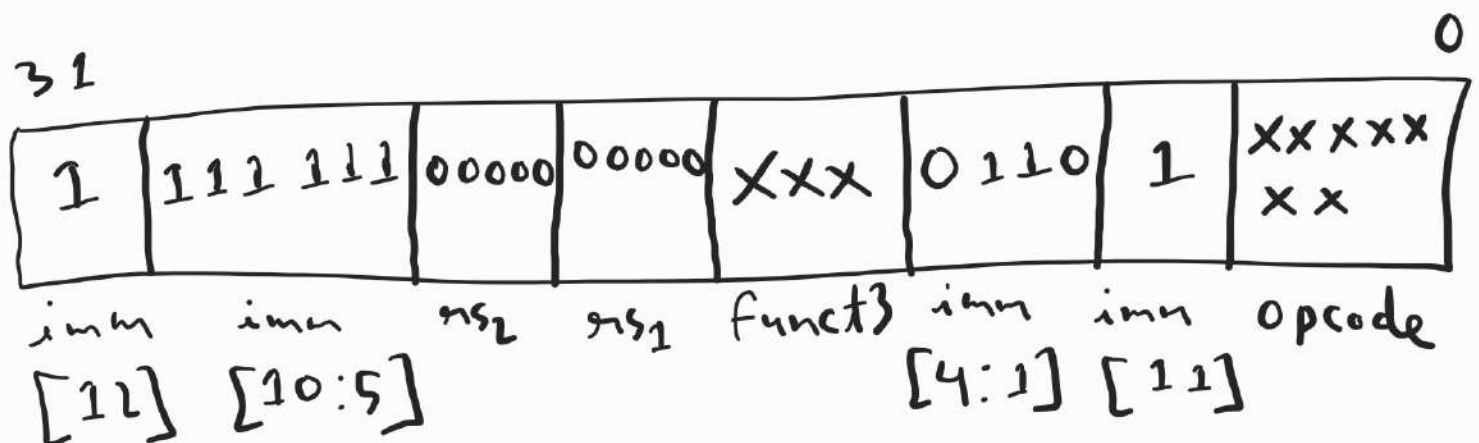
→ we perform sign extension,
here it is 0 cause we are
going forward with this
jump, so the offset/immediate will
be positive (For backward
jump it will be negative)

* We discard the last bit
after doing sign extension
because the immediate here
will always be a multiple
of 2 as per the rule 1,

hence the LSB will never have 1 in it which will make it an odd number, so we can discard the LSB as it will always be 0, hence does not need to be stored

* Another Example

beq x0, x0, Loop



* We are jumping from line 6 to line 1 here, which is a backward jump, we are going 5 lines backward/upward

* $5 \times 4 = 20$ { [It will be -20 since going upward]

no of lines going backward no of memory cells needed

$(20)_{10}$ in 12-bit

↓

0000 0001 0100

= 00000 0001 0100

11111 1110 1011 (1's)

+ 1

1	1111	1110	110	0	0	0	0	(2's)
↓	↓	└───┬───┘		└───┬───┘		└───┬───┘		
[12]	[11]	[10:5]		[4:1]		[0]		discarded

Machine code is given,
need to find the immediate

12	10:5				4:1	11	
1	111111	11000	01001	xxx	0110	1	xxx xxx x

i) Let's form the 12-bit number,

11 111 111 0110

ii) Figure out if the number is positive or not (using sign bit)

For negative we perform 2's complement

$$\begin{array}{r}
 1111 \quad 1111 \quad 0110 \\
 0000 \quad 0000 \quad 1001 \quad (1's) \\
 \quad \quad \quad +1 \\
 \hline
 0000 \quad 0000 \quad 1010 \rightarrow (10)_{10}
 \end{array}$$

iii) $PC \pm imm \times 2$

* The magnitude is 10,
the value itself is -10
since it was given negative
we multiply it with 2 because
we discard the LSB, which
makes the number to be
shifted 1-bit right, so
we adjust that by multiplying
2 with it to get the
actual value

Here,

If PC is 80020, then
the PC relative addressing

would be

$$80020 - 20 \text{ (Since the immediate was given negative)}$$
$$= 80000$$

↳ This is where we are jumping

This is the target location

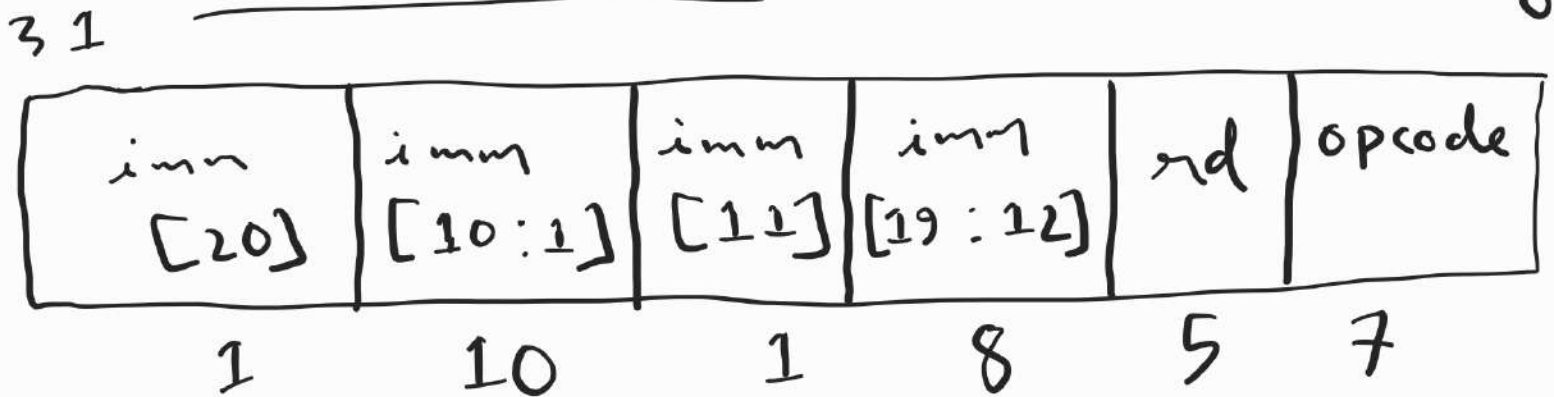
* There are another variant of RISC-V which has 16-bit machine code and that variant uses the formula as $\text{imm} \times 2$, they kept the formula same in the 32-bit variant as well by making necessary adjustments to keep both formulas generic

UJ Type Instruction

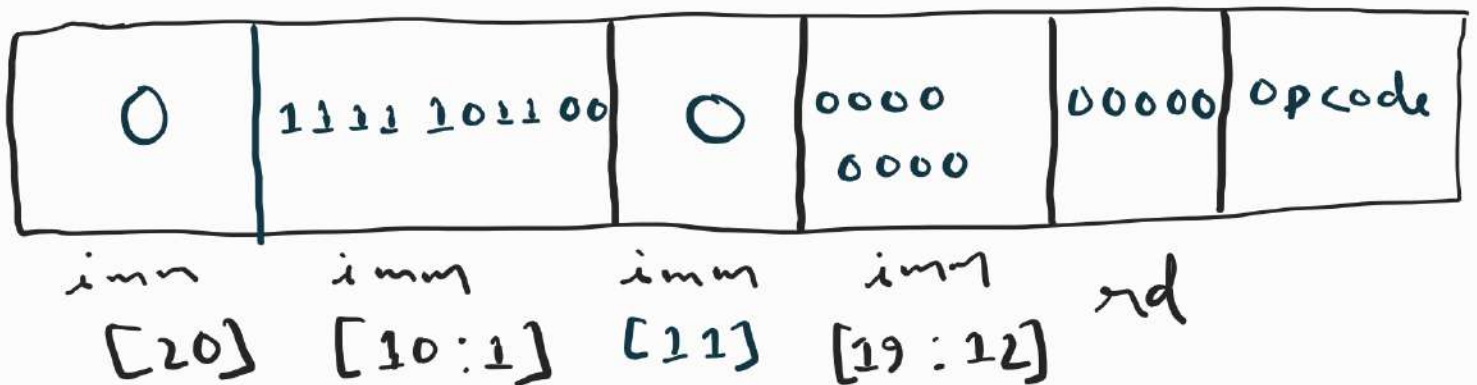
- * Only 1 instruction, Jal
- * The rules that are in SB type instruction are also applicable for Jal (Forward and backward moving, PC relative addressing etc.)
- * The immediate value is 20-bit in this format
- * This type is for unconditional jump

* This instruction allows for longer jump as immediate is 20-bit here

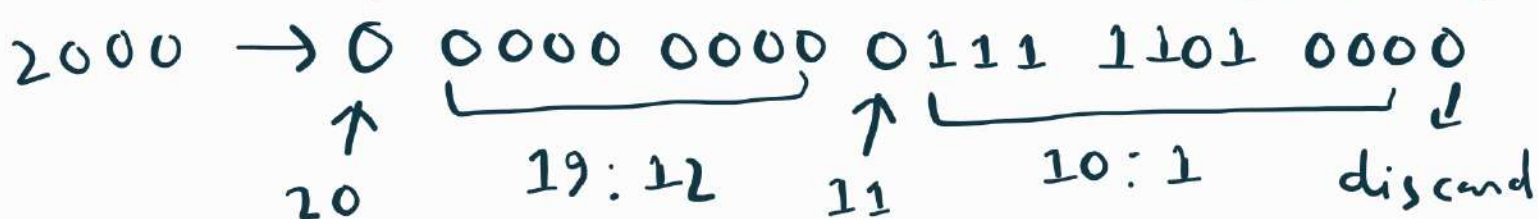
Format



Jal X0, 2000



→ Sign extension as positive immediate



Now,

Rest of the bits are converted as $(1000)_{10}$ as we discarded LSB which makes for 1-bit right shift (division by 2)

PC Relative addressing

$$\rightarrow PC \pm (1000 \times 2)$$

beq $X_{10}, X_0, L1$

replace with a pair of instructions that offers greater branching distance

Sol]:

bne $X_{10}, X_0, L2$

jal $X_0, L1$

L2:

* For much larger jump (32-bit),
lui $\rightarrow$ load address [31:12]
to a temp reg

* Then add [11:0] and use
jalr and jump to target

Procedure Calling

* Used to convert functions
to assembly

Steps

i) Put parameters in a place
where the procedure can
access them (X10 - X17)
argument registers

def calc(a, b, c):

parameters of function

$d = \text{calc}(\underbrace{1, 2, 3})$
 $\quad \quad \quad \hookrightarrow$ arguments

- ii) Transfer control to the procedure (function)

* when we are calling the function then the flow of the code is shifted to where the function is defined, that is transferring control to the function/procedure

we use JAL $\rightarrow$ jump and link instruction to do that

Syntax: JAL X₁, procedure Label
 ↓ ↓
 fixed jump destination

* Jump occurs to the label and the return address is stored in X_1 , we return to the location where we jumped from

```
calc(a,b,c): ←  
{  
  . . .  
  . . .  
main() {  
  1  
  2  
  3  
4 ~~~ = calc(1,2,3)
```

5 ↓
We return here again after procedure is done

* X_1 stores the location of

the instruction where we return after executing the procedure

calc(a,b,c): {

{ ... }

main() {

1 $\longrightarrow$ 0 (address in
2 $\longrightarrow$ 4 memory)

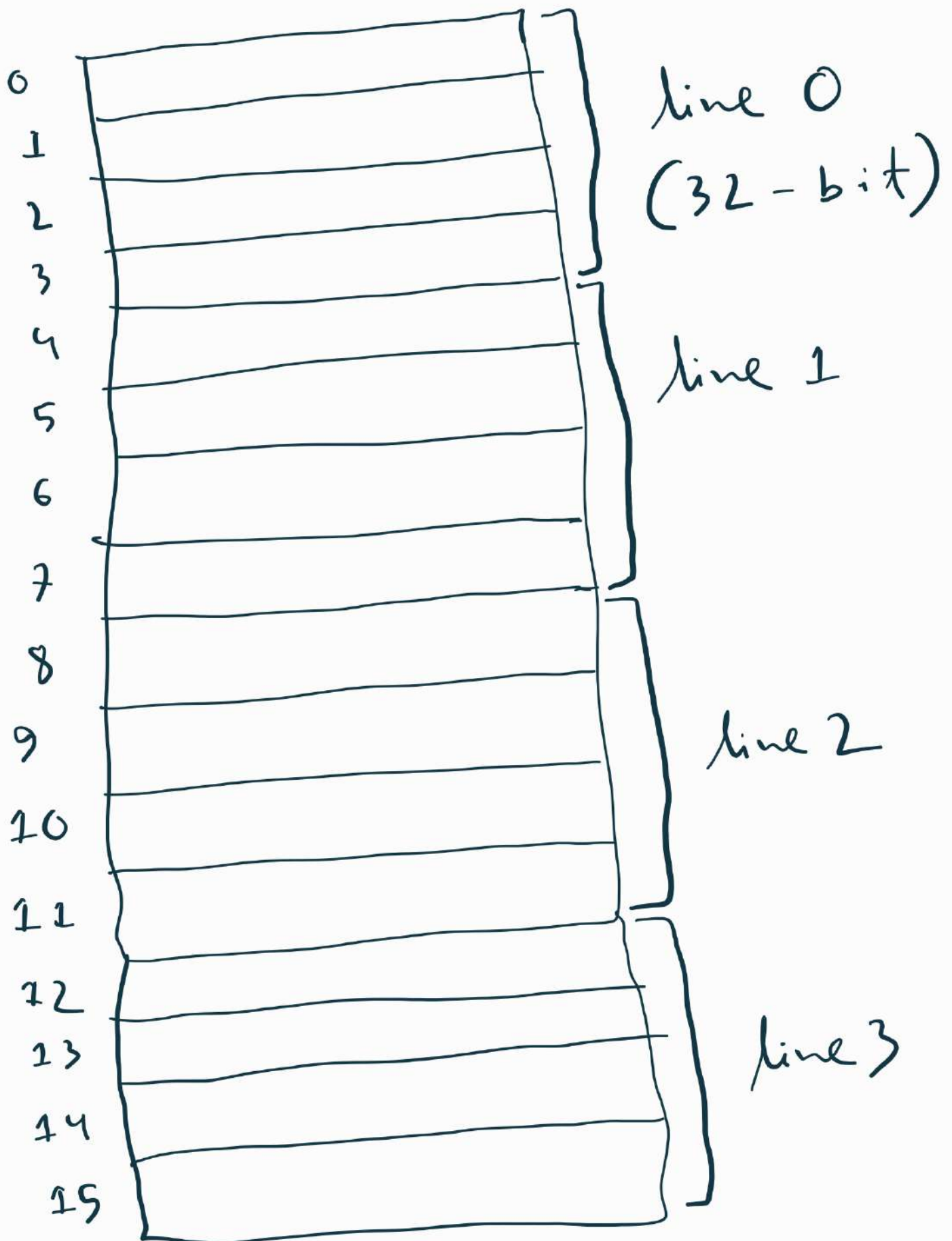
3 $\text{---} = \text{calc}(1,2,3) \rightarrow 8$

4 $\text{-----} \rightarrow 12$

* Program counter stores the memory location of the current instruction being executed

* PC will store 0 first when

it executes line 1, then it will store 4 when it executes line 2 and so on



* Let's assume there is an instruction $Jal\ x_1, label$ and PC had $\#0004$ in it. Now x_1 will store $4+4 = \#0008$ as that would be the instruction which are going to be executed once the procedure execution is done

* x_1 always stores the address we will return to after executing the function, which is current program counter + 4

Step iii) Acquire storage resources needed for procedure

* The registers that will be required to store necessary values should be made available

iv) Perform desired task which is executing the function

v) Put the result value in a place where the calling program can access it which are X_{10} and X_{11}

* If we store the return values elsewhere then the program can't read it

vi) Return control to the point of origin
↳ X_1

- * A procedure can be called from several points of a program
- * To return the control back to the origin (which is stored in X_1) we use another instruction,

JALR $\rightarrow$ Jump and link register

Syntax: JALR $X_0, \underline{O(X_1)}$
jump destination

- * Jalr has a memory location where we are jumping to instead of a label
- * They are jumping to $O(X_1)$ this location and storing

corresponding $PC+4$ in X_0 here

* Offset is 0 because the jump address is already in X_1

* We use `Jalr` to return to main method from the function using the address in X_1 , we do not need to store the current program counter as execution of the program is done, that is why X_0 is used here as destination as we do not need to store any return address here

* For jumping from one function to another function we need

to link (store the return addresses) but we do not need to link for returning to main method from function

* Main Method $\rightarrow$ Caller

The function $\rightarrow$ Callee
which is being called

Steps revisited :

* The calling program (caller) puts the parameter values in $X_{10} - X_{17}$

* Uses Jal X_1, abc to branch to procedure label abc (callee)

* Callee performs the calculations, places the results in the same

parameter registers ($x_{10} - x_{11}$)

* Returns control to the caller using jalr $x_0, 0(x_1)$

* Jal can be used for unconditional jump,

Jal x_0, Label

↳ Not storing return address here as 0 is hardwired in this register

* If the callee require any register which is already in use by the caller, then callee must restore the values that were contained before the

procedure is invoked, to do that the callee must store the previous value in the stack

main method $\rightarrow X_9, X_{12}$

function-1 $\rightarrow X_9$

↓
They can use it but first the function should store what is inside X_9 in stack and restore it later when they are done using X_9 so that main method can use the previous value

* Let's say we want to store 3 register data and

each register data is 64-bit

So,
we need $(3 \times 8) \rightarrow 24$ locations

* Stack pointer (SP) stores
the top address of the
stack, X_2 is the stack
pointer register

First we need to decrease
the stack pointer by the
no of locations needed to
store the desired data, in
this case which is 24,

Addi $X_2, X_2, -24$

Then we store the value

* If the callee wants to use a saved register ($X_{18}-X_{27}$) then they need to store the value in stack first, doesn't matter what they have, same is not necessary for temporary registers

* Stack is last in first out, so let's say callee needs to store values for X_5 , X_6 and X_{20} in stack,

Code for loading and storing the values are given below considering for double word array

addi x2, x2, -24 $\rightarrow$ 3 reg values

sd x5, 16(x2) $\rightarrow$ stack is a memory location

sd x6, 8(x2)

sd x20, 0(x2)

⋮

ld x20, 0(x2) $\rightarrow$ last in first out

ld x6, 8(x2)

ld x5, 16(x2)

addi $x_2, x_2, 24$

* loads last added data first as stack is last in first out, also adjust the stack pointer at the end